LangSmith LLM Gateway

A detailed, prioritized guide to routing, governance, security, observability, and production rollout

90 min	5	4
fast learning path	core policy types	hands-on labs

Audience: engineers, solution architects, platform teams, and technical leaders evaluating or implementing the LangSmith LLM Gateway.

Based on current LangChain documentation. The LLM Gateway is currently documented as beta.[1]

How to use this guide

This guide is ordered by learning value, not by the order of the documentation pages. Start with the request path, API formats, identity, and tracing. Add policies next. Study custom endpoints and direct provider access only after the standard path is clear.

> **Recommended study ratio**
> **30% reading, 70% doing.** Complete the small labs as you go. A gateway is much easier to understand after you have sent one request, inspected its trace, and deliberately triggered one policy.

Learning priorities

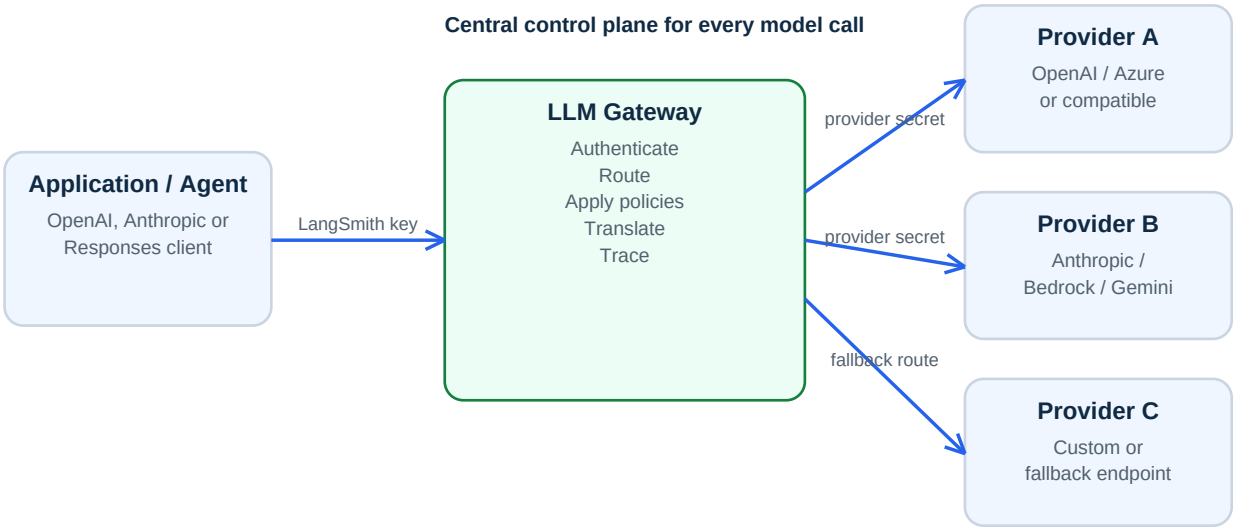
Priority	Concepts	Outcome
P0 - Core	Mental model, request lifecycle, endpoint versus model ID, credentials, first request, traces	You can explain and debug a normal gateway call.
P1 - Production	Spend, rate, model access, data policy, fallbacks	You can govern cost, throughput, model choice, sensitive data, and availability.
P2 - Advanced	Custom configurations, direct access, regional/BYOC/self-hosted, coding-agent rollout	You can handle nonstandard providers and enterprise deployment patterns.

Table of contents

1. Mental model and boundaries
2. Request lifecycle
3. Identity and credentials
4. API formats and model routing
5. First-call lab
6. Traces and access control
7. Governance policies
8. Data protection and retention
9. Fallbacks and resilience
10. Custom configurations and direct access
11. Deployment patterns
12. Production architecture patterns
13. Troubleshooting
14. Rollout checklist and study plan
15. Source map

1. Mental model and product boundaries

The LLM Gateway is a proxy and governance layer between an application and one or more model providers. The application authenticates to LangSmith, while the gateway resolves the provider route and upstream credential, applies policies, and records a trace.[1][3]



What it is
A single controlled entrance for model traffic: centralized credentials, routing, API translation, tracing, cost controls, model controls, sensitive-data handling, and failover.

What it is not
It is not an agent runtime, planner, tool executor, vector database, prompt-management system, or evaluation framework. Those systems can call through the gateway, but the gateway does not replace them.

Four ideas to remember

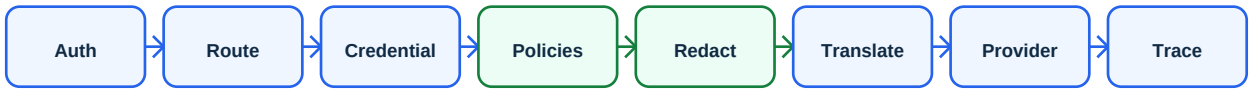
Idea	Meaning
One key, multiple providers	Developers use a workspace-scoped LangSmith API key rather than keeping each provider key locally.
One client format, multiple models	A client can use Chat Completions, Messages, or Responses while selecting the upstream route with the model ID.
Central governance	Policies are evaluated at the gateway, so controls are consistent when traffic actually flows through it.
Built-in observability	Every proxied call produces a gateway trace with usage, cost, routing, status, latency, and policy metadata.

The trust boundary

A gateway only governs calls that pass through it. If developers retain direct provider credentials, they can bypass the gateway. Centralizing upstream provider keys in workspace secrets and withholding those keys from callers is therefore part of the control design, not just setup convenience.[11]

2. Follow one request through the gateway

The complete request path is the backbone of the product. Learn it before individual features.



Response returns in the client-selected API format; redaction placeholders are restored when applicable.

Stage	Question answered	Failure signal
1. Authenticate	Is the LangSmith API key valid and workspace-scoped?	401 for missing or invalid authentication.
2. Authorize	Does the attached role contain gateway:invoke and workspaces:read?	403 for insufficient permission.
3. Resolve route	Which provider/model or saved configuration does the model ID identify?	400 for unavailable or malformed IDs.
4. Resolve credential	Which workspace provider secret or Gateway Credits credential should be used?	Provider/configuration error if unavailable.
5. Evaluate policies	Are spend, rate, model-access, and data rules satisfied?	402, 403, or 429 depending on policy.
6. Translate request	Can the selected client format be represented for the provider?	400 when fields cannot be translated.
7. Call provider	Does the upstream request succeed or stream?	Provider status or transport error.
8. Return and trace	Can the response be translated, placeholders restored, and metadata recorded?	Final response plus a gateway trace.

Direct access changes only part of the flow
Provider-specific direct paths skip request and response translation. Authentication, upstream credential resolution, policy evaluation, and tracing still apply.[3][13]

Control plane versus data plane

Control-plane configuration includes provider secrets, roles, policies, fallback chains, and model configurations. **Data-plane activity** is the live model request moving through authentication, enforcement, provider invocation, and tracing. Keeping these concepts separate helps diagnose whether a failure is configuration-time or request-time.

3. Identity, credentials, and permissions

There are two distinct authentication relationships. Conflating them is a common source of setup errors.

Hop	Credential	Purpose	Owner
Application -> Gateway	Workspace-scoped LangSmith API key	Identifies and authorizes the caller.	Platform or workspace administrator
Gateway -> Provider	Workspace Provider Secret	Authenticates to a provider under your provider account.	Administrator
Gateway -> Hosted model	Gateway Credits credential	Uses a LangChain-managed upstream credential.	LangChain/LangSmith account

Required permissions

Action	Permission	Typical role
Invoke gateway	gateway:invoke + workspaces:read	Workspace admin, or a custom role with both permissions
Manage policies	organization:manage	Organization admin
View gateway traces	projects:read + runs:read	Built-in workspace roles include trace-reading permissions
View audit logs	organization:manage	Organization admin; audit-log availability depends on plan

Least-privilege recommendation
Create a custom workspace role containing only the required gateway permissions when RBAC is available. Do not grant full workspace administration merely to permit model calls.[5][11]

Admin setup sequence

1. Add only the Provider Secrets the workspace needs. Hosted Gateway Credits models do not require your own provider secret.
2. Grant users or service identities a role with **gateway:invoke** and **workspaces:read**.
3. Create workspace-scoped service keys. Organization-scoped keys are not supported for invocation.
4. Add policies after the first request works, then retest each control deliberately.
5. Verify with a real call and confirm that the trace appears in the expected workspace gateway project.

4. API formats and model routing

The endpoint and the model ID perform different jobs. The endpoint selects what the client sends and receives. The model ID selects the upstream route.[4]

Client/API format	Cloud base URL	Request path
OpenAI Chat Completions	https://gateway.smith.langchain.com/v1	POST /chat/completions
OpenAI Responses	https://gateway.smith.langchain.com/v1	POST /responses
Anthropic Messages	https://gateway.smith.langchain.com	POST /v1/messages

Model identifiers

Pattern	Meaning	Example shape
provider/model	Bring-your-own-key model routed through a configured provider	openai/ or anthropic/
azure/deployment	Azure deployment selected by deployment name	azure/
hosted slug	Gateway Credits model	A currently supported hosted model slug
custom/name	Saved model configuration	custom/

Use live discovery
Call **GET /v1/models** to discover currently available standard models for the workspace. Saved custom configurations do not appear in this list, so their exact names must be distributed separately.[4][12]

Translation behavior

- If the provider supports the client-selected format natively, the gateway preserves that format.
- Otherwise, the gateway translates the request and response, including streaming responses.
- A cross-provider request is therefore possible: for example, an Anthropic Messages client may target an OpenAI model.
- Fields without an equivalent in the target provider format can be rejected. Translation is interoperability, not a guarantee of perfect semantic equivalence.
- Use direct model access when provider-native semantics are a hard requirement.

5. Lab: make, observe, and modify your first call

Goal: prove that identity, routing, upstream credentials, and tracing all work before adding governance.

Step A - Set the key

```
export LANGSMITH_API_KEY="<workspace-scoped-key>"
```

Step B - List available models

```
curl https://gateway.smith.langchain.com/v1/models \
  -H "Authorization: Bearer $LANGSMITH_API_KEY"
```

Step C - Send a Chat Completions request

```
curl https://gateway.smith.langchain.com/v1/chat/completions \
  -H "Authorization: Bearer $LANGSMITH_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model":"<provider>/<model>","messages":[{"role":"user","content":"Explain LLM gatewa
ys in three bullets."}]}'
```

Step D - Inspect the trace

- Open LangSmith and enter the tracing project named **gateway** in the key's workspace.
- Confirm status, model, token counts, cost, latency, route, and policy metadata.
- Notice whether request/response content is absent. Content logging is off by default unless a matching data policy enables it.[11]

Step E - Prove model routing

Keep the endpoint and payload format the same, but replace the model ID with another available provider-prefixed model. If the call succeeds, you have demonstrated that the model ID controls the upstream route while the endpoint controls the client format.

> **Completion criterion**
> You can explain why a 200 response validates four things at once: gateway reachability, API-key validity, role permissions, and availability of the selected upstream credential.[2][5]

6. Tracing, visibility, and access control

Every gateway-proxied call is written to a project named **gateway** in the workspace associated with the invoking key.[11]

Trace aspect	Default	When changed
Metadata	Recorded	Always includes operational metadata such as status, model, latency, usage, cost, and policy evaluation.
Request/response bodies	Off	A matching data policy can enable Trace content.
Trace quota	Metadata-only traces are not billed against trace quota	Content-bearing traces count toward plan trace volume.
Visibility	Workspace project permissions	Restrict with workspace separation or project-level ABAC where available.

Policy evidence in traces

Gateway spans record which policies matched, passed, or were violated. Redaction pipelines also write rule-match metadata. This creates an operational trail for answering questions such as: Was the request blocked? Which rule fired? Was spend or rate state involved? Was sensitive material replaced before provider delivery?

LangSmith Engine and auditability

Policy events can surface as issues in LangSmith Engine for triage. Administrators can move from the issue to the associated trace, diagnose the cause, and decide whether to change the application or the policy. Administrative changes and gateway invocations are also audit-relevant events; documented audit-log access depends on role and plan.[11]

> **Privacy design decision**
> Trace-content logging and provider redaction solve different problems. Redaction protects selected outbound content from the provider; trace-content settings decide whether bodies are stored in LangSmith gateway traces.

7. Governance policies

Gateway governance is easier to reason about as five separate control questions.

Policy/control	Question	Typical block/result
Spend	May this subject spend more in the current window?	402 when the cap is exceeded
Rate	May this subject consume more requests or tokens now?	429 plus Retry-After
Model access	May this subject call this provider/model?	403 when not permitted
Data policy	What should be logged, detected, or redacted?	Redaction; timeout may allow or block
Fallback	If the selected route fails, which backup should be tried?	Retries ordered candidates on configured triggers

7.1 Spend policies

A spend policy defines a USD cap for an organization, workspace, API key, or user over an hourly, daily, weekly, or monthly window. The gateway tracks spend in real time and blocks a request that would push the subject past its cap. The violation is traced.[6]

- Start with a low daily cap on development API keys.
- Add workspace and organization caps as broader safety nets.
- Use a custom X-Gateway-* header when one application key serves multiple downstream customers and each customer needs an independent allowance.
- Remember that a cap is a hard control, not an alert. Design the caller to handle HTTP 402 intentionally.

7.2 Rate-limit policies

Rate policies restrict request count or provider-reported total tokens over minute or hour windows. A single policy may contain multiple nonduplicate metric/window combinations.[7]

Dimension	Example
User	Each developer receives 100 requests per minute.
Workspace	The R&D; workspace receives one million tokens per hour.
API key	A group of support-agent keys shares a request allowance.
Default versus granular	A default applies broadly; a named granular policy overrides the default for its subject.

Client behavior
On 429, honor the **Retry-After** header. Unbounded immediate retries make rate limiting worse and can also increase cost when retries reach providers.

7.3 Model-access policies

A model-access policy is an allowlist of providers and models. If no model-access policy applies, models are available by default. A matching policy can allow all models from a provider or only selected models.[8]

Scope precedence

API key -> user -> workspace -> organization. The most specific matching tier replaces the broader tier entirely. When several policies match at the same tier, a model must be permitted by all of them.

> **Example**
> The organization allows OpenAI and Anthropic. A specific production API-key policy allows only OpenAI and Gemini. For that key, the key-level policy replaces the organization policy: OpenAI and Gemini are allowed; Anthropic is not.[8]

7.4 Suggested policy stack

Environment	Spend	Rate	Model access	Data
Developer	Low daily key cap	Per-user request cap	Approved economical models	Secrets detection; content off by default
Staging	Workspace daily cap	Workload-shaped request/token cap	Production candidate models	Test the intended PII/secrets rules
Production	Layered key + workspace + org caps	API-key and customer-aware limits	Explicit approved models; narrow exceptions	Documented retention and timeout behavior

Policy testing method

1. Create the policy on a disposable test key or isolated workspace.
2. Send one request that should pass and inspect its policy metadata.
3. Send one request that should violate the policy and record the exact HTTP status and error body.
4. Verify that the expected trace or issue exists.
5. Test application behavior: fallback, backoff, user-facing message, or escalation.
6. Only then expand the policy scope.

8. Data retention and sensitive-data protection

A data policy covers two independent areas: retention in gateway traces and protection of outbound provider requests.[9]

Area	Controls	Key question
Data retention	Trace content on/off	Should request and response bodies be stored in the gateway trace?
Data protection	PII/secrets detection, scan timeout, timeout action	What must be replaced before the request reaches the model provider?

Redaction lifecycle

1. The gateway scans covered outbound request content.
2. Detected values are replaced with typed placeholders before provider delivery.
3. The provider receives the redacted payload and may repeat those placeholders.
4. For a successful response, the gateway restores matching placeholders before returning data to the caller.
5. If trace content is enabled, the gateway trace reflects what the provider saw: the redacted representation.

Detection categories

Type	Examples
Rule-based PII	Email, formatted US phone numbers, US Social Security numbers
Model-based PII	Names, locations, nationality/religious/political groups
Secrets	LangSmith, AWS, GitHub, GitLab, OpenAI, Anthropic, GCP, Azure, Google OAuth, Slack, Datadog, PyPI, npm, private keys, JWTs, Stripe, SendGrid

Security boundaries you must not miss

Not covered or special case	Consequence
Provider-generated sensitive output	The model response itself is not scanned for newly generated sensitive data.
System/developer prompts	These are skipped by the documented gateway scanner.
Some tool-call representations	Assistant tool-call metadata and arguments arriving as one JSON string may be skipped; structured tool-call arguments are covered.
Provider errors	For status 400 or above, placeholders may pass through without restoration.
Direct LangSmith traces	Traffic that bypasses the gateway is not protected by gateway redaction.
Scanner error	Scanner failures generally fail open; only configured scan timeout behavior can block on timeout.
Platform-wide privacy	Gateway redaction alone does not prevent sensitive data from entering every LangSmith ingestion path.

Risk decision
If the requirement is "sensitive data must never leave the application boundary," gateway redaction alone is not sufficient. Add application-side validation/redaction, strict content boundaries, output controls, and an architecture-level review.

9. Fallbacks and resilience

A fallback chain retries a request against ordered backup models when the primary returns configured trigger statuses or a transport error.[10]

Component	Meaning
Primary	The provider/model or saved configuration named by the incoming request.
Candidates	One to five ordered direct models or saved model configurations.
Triggers	Configured upstream status codes, commonly 429 and selected 5xx responses.
Termination	Stop when a candidate succeeds, returns a nontriggering status, or the chain is exhausted.
Client contract	Return the final response in the API format used by the caller.

Example chain

```
Primary: anthropic/<primary-model>
  trigger: 429, 500, 502, 503, 504
Fallback 1: openai/<backup-model>
Fallback 2: custom/<self-hosted-config>
```

Fallback design checklist

- **Capability parity:** Does every candidate support required tool calls, structured output, context length, modalities, and streaming?
- **Semantic parity:** Will instructions and safety policies behave acceptably across providers?
- **Cost:** Could a backup be significantly more expensive than the primary?
- **Latency:** What is the worst-case delay after several failed attempts?
- **Residency and compliance:** Does every provider/region satisfy the same constraints?
- **Observability:** Can operations identify which route ultimately answered?
- **Idempotency:** Could retrying a request cause duplicate external actions in an agent workflow?

> **Fallbacks are not quality-neutral**
> The gateway can translate formats, but it cannot guarantee equivalent reasoning, tool behavior, safety, latency, or price across models. Evaluate the complete chain with representative tasks.

Lab: prove failover

1. Create a chain with a primary and one backup model.
2. Configure a trigger you can safely reproduce in a test environment.
3. Send a request using the primary model ID.
4. Confirm the backup answered and inspect routing metadata in the trace.
5. Repeat with a tool-calling or structured-output request, not just plain text.

10. Custom model configurations and direct access

10.1 Saved model configurations

A saved configuration is a named, reusable provider connection definition. When the request uses **custom/<name>**, the gateway resolves the configured provider/model, API-key secret, base URL, and supported connection settings.[12]

Use a configuration when...	Do not assume...
You need a custom OpenAI-compatible or Anthropic-compatible endpoint.	That it appears in GET /v1/models; custom names must be shared separately.
The provider-prefixed model form cannot represent the target endpoint.	That saved sampling parameters are applied. Pass temperature, max tokens, and similar values in the request.
You want the configuration as a primary or candidate in a fallback chain.	That model-access policies automatically permit saved configurations.

10.2 Direct model access

Direct model access exposes provider-specific paths and preserves the provider-native request/response contract. The gateway still authenticates, resolves secrets, enforces policies, and traces the call, but does not perform cross-provider format translation.[13]

Choose standard access	Choose direct access
You value one normalized client contract across providers.	You require exact provider-native fields or response semantics.
You want easy cross-provider fallback and portability.	Translation rejects or alters a feature you require.
Your application already uses Chat Completions, Responses, or Messages.	Your provider SDK must speak directly to its native path.

Default rule
Prefer the standard API. Move to direct access only for a documented provider-native requirement; otherwise you give up some portability without gaining value.

11. Deployment patterns and regional behavior

The gateway is documented for LangSmith Cloud regions, BYOC data planes, and self-hosted LangSmith. API formats, model IDs, policies, and tracing concepts remain consistent; hostname and path details change.[3]

Deployment	Traffic location	Path characteristic	When to consider
LangSmith Cloud	LangSmith regional gateway	Standard /v1 paths	Fastest managed setup
BYOC	Customer data plane/VPC	Gateway paths use a /gateway prefix	Data-plane control inside your cloud boundary
Self-hosted	Your LangSmith deployment	Gateway paths use a /gateway prefix; admin enablement required	Maximum infrastructure control

Documented Cloud hostnames

Region	Hostname
GCP US	gateway.smith.langchain.com
GCP EU	eu.gateway.smith.langchain.com
GCP APAC	apac.gateway.smith.langchain.com
AWS US	aws.gateway.smith.langchain.com

Configuration portability
Do not hard-code a US gateway URL into reusable libraries. Make the gateway base URL explicit configuration so the same application can target a regional, BYOC, or self-hosted deployment.

Coding-agent rollout

Claude Code, Codex, Gemini CLI, and Deep Agents Code can be routed through the gateway. Treat these as additional clients, not a different gateway architecture. Plan key distribution, least-privilege roles, provider authentication, model-access policy, spend/rate limits, and trace visibility before organization-wide rollout.

12. Production architecture patterns

Pattern A - Shared enterprise gateway

- One or more workspaces align with organizational boundaries.
- Provider keys stay in workspace secrets; application teams receive LangSmith service keys.
- Organization policy supplies the broad baseline; workspace and API-key policy creates controlled exceptions.
- Separate gateway projects and access rules protect trace visibility.

Pattern B - Multi-tenant SaaS

- A service key invokes the gateway on behalf of multiple customers.
- A custom X-Gateway-* header partitions spend or rate state per downstream customer.
- Application authentication still owns customer identity; the header must be derived from trusted server-side context, not accepted blindly from an end user.
- Customer-facing errors distinguish capacity, budget, and model-availability failures.

Pattern C - Resilient agent

- Primary and fallback models are evaluated for tool and structured-output compatibility.
- Agent operations are designed for idempotency because retries can duplicate side effects.
- Trace metadata records the actual route, allowing reliability and quality to be compared by provider.
- Spend caps account for worst-case fallback cost, not only the cheapest primary route.

Pattern D - Sensitive-data workload

- Application-side validation and minimization happen before gateway invocation.
- Gateway detection provides an additional outbound control, not the only control.
- Trace content stays off unless a documented operational need justifies storage.
- The team explicitly reviews skipped content surfaces, timeout behavior, fail-open behavior, provider errors, and generated-output handling.
- Data residency and provider selection are enforced independently of redaction.

Architecture review questions

Area	Question
Identity	Which human or service identity does each LangSmith key represent?
Bypass	Who still possesses direct provider credentials?
Scope	Which workspace owns the key, policies, secrets, and gateway traces?
Failure	What should users see for 402, 403, 429, provider errors, and exhausted fallbacks?
Data	Which fields can enter prompts, traces, provider logs, and application logs?
Quality	How are fallback routes evaluated against the primary?
Operations	Who receives and resolves policy violations or abnormal spend?

13. Troubleshooting playbook

Diagnose in request-path order. Avoid changing several controls simultaneously.

Symptom/status	Likely meaning	Check next
400 Bad Request	Malformed request, unavailable/malformed model ID, or untranslatable field	Validate endpoint format, model ID, and provider-specific fields. Try GET /v1/models.
401 Unauthorized	Missing or invalid LangSmith API key	Confirm Authorization header and that the key is current.
403 Forbidden	Missing gateway permission or model blocked	Check gateway:invoke/workspaces:read and model-access policy.
402 Payment Required	Spend policy blocked the request	Inspect policy name, scope, current window, and trace metadata.
429 Too Many Requests	Gateway or provider rate limit	Honor Retry-After; determine whether the limit is gateway-side or upstream.
Provider-prefixed models absent	Provider may not be configured or its catalog unavailable	Check Provider Secrets and workspace configuration.
Custom model not listed	Expected behavior	Use the exact shared custom/; custom configs are not returned by GET /v1/models.
Trace has no body	Trace content is off by default	Review matching data policy; do not enable bodies casually.
Placeholder appears in error	Provider returned status 400+	Documented behavior: provider error bodies may not be rehydrated.
Fallback never triggers	Status is not configured, chain does not match primary, or policy blocked before provider call	Inspect trace route/policy metadata and chain definition.

Debug sequence

1. Reduce the request to a minimal prompt without tools, structured output, or streaming.
2. Verify the base URL and endpoint match the client format.
3. Verify authentication and role permissions.
4. List available models and use one known route.
5. Check the provider secret or Gateway Credits availability.
6. Inspect policy results in the gateway trace.
7. Reintroduce translation-sensitive fields one at a time.
8. Finally test fallback, streaming, tools, and structured output.

14. Production rollout checklist

Phase 1 - Foundation

- [ ] Select Cloud region, BYOC, or self-hosted deployment.
- [ ] Define workspace boundaries and trace visibility.
- [ ] Store only required provider credentials.
- [ ] Create least-privilege invocation roles and workspace-scoped service keys.
- [ ] Send a minimal request and verify the gateway trace.

Phase 2 - Governance

- [ ] Add development spend and rate limits first.
- [ ] Define approved providers and models.
- [ ] Decide whether gateway trace bodies should ever be stored.
- [ ] Configure PII/secrets rules and document scanner boundaries.
- [ ] Test one pass and one deliberate violation per policy.

Phase 3 - Resilience

- [ ] Design fallback candidates for capability, cost, latency, and compliance parity.
- [ ] Test provider error, rate-limit, timeout, and chain-exhaustion behavior.
- [ ] Ensure application side effects are idempotent or guarded.
- [ ] Implement status-aware retry/backoff and user-facing error messages.

Phase 4 - Operations

- [ ] Build views or alerts for cost spikes, latency, provider errors, and policy violations.
- [ ] Assign owners for policy tuning and incident response.
- [ ] Rotate keys and provider secrets through a documented process.
- [ ] Re-evaluate model access and fallbacks whenever providers or model versions change.
- [ ] Review beta-release changes before depending on undocumented behavior.

90-minute fast-study plan

Time	Activity	Proof of understanding
0-30 min	Overview, request lifecycle, quickstart, first trace	Explain endpoint vs. model ID and both credential hops.
30-60 min	Spend, rate, model access, trace-content behavior	Predict 402, 403, and 429 outcomes.
60-90 min	Data policy boundaries and fallback chain	Explain what provider, trace, and application see; demonstrate failover.

One-page reference

Core formula

Gateway call
Client format is selected by the endpoint + **upstream route** is selected by the model ID + **caller identity** comes from the workspace-scoped LangSmith key + **provider identity** comes from a Provider Secret or Gateway Credits.

Need	Feature
Control total cost	Spend policy
Control burst/throughput	Rate-limit policy
Restrict providers/models	Model-access policy
Avoid selected outbound secrets/PII	Data-protection policy
Control trace body storage	Data-retention setting
Survive provider/model errors	Fallback chain
Reach custom endpoint	Saved model configuration
Preserve provider-native API	Direct model access
Explain a call	Gateway trace

Status-code mnemonic

Code	Remember
400	Request/translation
401	Authentication
402	Spend
403	Permission or model access
429	Rate limit
5xx	Provider/service failure; possible fallback trigger

Final mental model
The gateway centralizes **who may call**, **what they may call**, **how much they may consume**, **what data may leave**, **where traffic goes when a provider fails**, and **how every decision is observed**.

15. Source map

Inline bracketed numbers correspond to the official LangChain documentation pages below. URLs are shown in full for durable reference.

[1] **LLM Gateway overview**
https://docs.langchain.com/langsmith/llm-gateway

[2] **LLM Gateway quickstart**
https://docs.langchain.com/langsmith/llm-gateway-quickstart

[3] **How the LLM Gateway works**
https://docs.langchain.com/langsmith/llm-gateway-how-it-works

[4] **API formats**
https://docs.langchain.com/langsmith/llm-gateway-api-formats

[5] **Administrator setup**
https://docs.langchain.com/langsmith/llm-gateway-admin-setup

[6] **Spend policies**
https://docs.langchain.com/langsmith/llm-gateway-spend-policies

[7] **Rate limit policies**
https://docs.langchain.com/langsmith/llm-gateway-rate-limit-policies

[8] **Model access policies**
https://docs.langchain.com/langsmith/llm-gateway-model-access-policies

[9] **Data policy**
https://docs.langchain.com/langsmith/llm-gateway-data-policy

[10] **Model fallbacks**
https://docs.langchain.com/langsmith/llm-gateway-fallbacks

[11] **Traces and access control**
https://docs.langchain.com/langsmith/llm-gateway-access

[12] **Custom model configurations**
https://docs.langchain.com/langsmith/llm-gateway-model-configurations

[13] **Direct model access**
https://docs.langchain.com/langsmith/llm-gateway-direct-model-access

---

Prepared as a learning guide. Confirm current availability, entitlement, model names, pricing, and beta behavior in the live documentation before production decisions.